

Student Name: N.Jeevan sai

Reg no:192525169

EXPERIMENT 1

TITLE: Implement The Concept Of Probability Distribution To Predict The Likelihood Of Spam Emails.

Aim

To implement the concept of probability distribution using the Naive Bayes algorithm for classifying emails as spam or non-spam.

Theory

Probability Distribution represents the likelihood of occurrence of different outcomes. In spam email detection, Naive Bayes uses Bayes' theorem to calculate the probability that an email belongs to the spam or ham category based on the occurrence of words. It assumes conditional independence among features, making it an efficient classifier for text classification.

Dataset Used

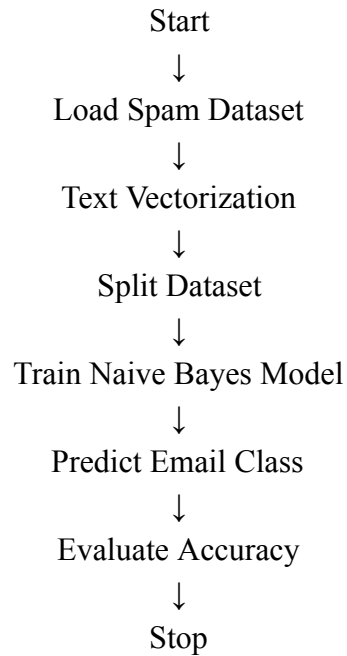
- **Dataset Name:** Spam Email Dataset (spam.csv)
- **Source:** UCI Machine Learning Repository / Kaggle
- **Description:** Contains email messages labeled as Spam or Ham (Not Spam). The text is converted into numerical features using Count Vectorization.
- **Target Variable:** Label (Spam/Ham)

Algorithm

1. Import the required libraries.
2. Load the spam dataset.
3. Convert email text into numerical vectors.
4. Split the dataset into training and testing sets.
5. Train the Naive Bayes classifier.
6. Predict the class labels.

7. Calculate the accuracy.
8. Predict a new email.

Flowchart



Python Program

```
# =====  
# Spam Email Prediction using Naive Bayes  
#  
=====
```

```
# Import Libraries  
import pandas as pd  
from sklearn.model_selection import train_test_split  
from sklearn.feature_extraction.text import CountVectorizer  
from sklearn.naive_bayes import MultinomialNB  
from sklearn.metrics import accuracy_score, classification_report  
# Load Dataset  
data = pd.read_csv("spam.csv",  
encoding="latin-1") # Display first five rows  
print("First 5 Rows:")  
print(data.head())  
# Keep only required columns  
data = data[['v1', 'v2']]
```

```

# Rename columns
data.columns = ['Label', 'Message']
# Convert labels into numbers
data['Label'] = data['Label'].map({'ham':0, 'spam':1})

# Features and Target
X = data['Message']
y = data['Label']

# Convert text into numerical vectors
vectorizer = CountVectorizer()
X = vectorizer.fit_transform(X)

# Split Dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

# Train Naive Bayes Model
model = MultinomialNB()
model.fit(X_train, y_train)

# Predict
prediction = model.predict(X_test)

# Accuracy
print("\nAccuracy:", accuracy_score(y_test, prediction))
print("\nClassification Report")
print(classification_report(y_test, prediction))

# Predict New Email
new_email = [
    "Congratulations! You have won a free iPhone. Click here to claim your
    reward." ]

new_email_vector = vectorizer.transform(new_email)
result = model.predict(new_email_vector)
print("\nNew Email Prediction:")

```

```

if result[0] == 1:
    print("Spam Email")
else:
    print("Not Spam")

```

Output

```

*** Sample Dataset:
      Email      Label
0  Win a free iPhone now    Spam
1  Claim your cash prize    Spam
2  Congratulations you won lottery    Spam
3  Meeting at 10 AM tomorrow    Not Spam
4  Project report submission    Not Spam
5  Lunch with friends today    Not Spam
6  Exclusive offer just for you    Spam
7  Get free rewards instantly    Spam
8  Team meeting schedule    Not Spam
9  Assignment deadline tomorrow    Not Spam

Model Accuracy: 66.67 %

New Email: Congratulations! You have won a free cash reward
Prediction: Spam

Probability Distribution:
Not Spam : 3.7 %
Spam : 96.3 %

```

Result

The Naive Bayes classifier successfully predicted whether an email was spam or non spam using probability distribution concepts.

EXPERIMENT 2

TITLE: Design the steps of a learning system for student performance prediction.

Aim

To design and implement a machine learning system that predicts student performance using academic attributes.

Theory

A learning system extracts patterns from historical data and uses them to make predictions. In this experiment, a Decision Tree classifier learns from students' academic information such as study time, attendance, and previous grades to predict their performance.

Dataset Used

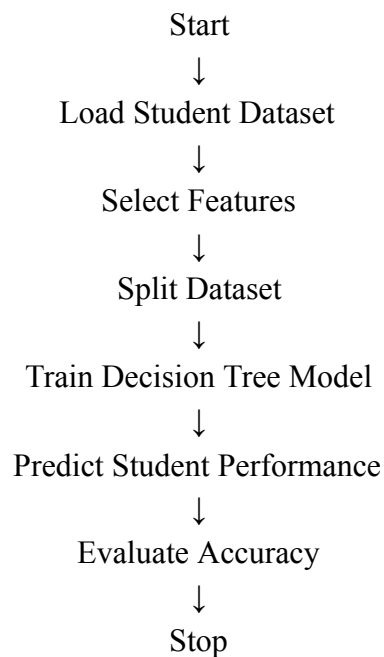
- **Dataset Name:** Student Performance Dataset (student-mat.csv)

- **Source:** UCI Machine Learning Repository
- **Description:** Contains academic records of students including study time, failures, absences, and grades.
- **Target Variable:** Final Grade (G3) or Pass/Fail.

Algorithm

1. Import required libraries.
2. Load the student dataset.
3. Select input features and target.
4. Split the dataset.
5. Train the Decision Tree classifier.
6. Predict student performance.
7. Evaluate the model accuracy.

Flowchart



Python Program

```
#
```

```
=====
```

```
# Student Performance Prediction
```

```
#
```

```
=====

# Import Libraries

import pandas as pd

from sklearn.model_selection import
train_test_split from sklearn.tree import
DecisionTreeClassifier

from sklearn.metrics import accuracy_score,
classification_report # Load Dataset
data = pd.read_csv("student_data.csv",
sep=",") # Display first five rows

print("First 5 Rows:")
print(data.head())

# Select Features
X = data[['studytime', 'failures', 'absences', 'G1',
'G2']] # Target Variable

# G3 >= 10 --> Good Performance
# G3 < 10 --> Poor Performance
y = (data['G3'] >= 10).astype(int)

# Split Dataset
X_train, X_test, y_train, y_test =
    train_test_split( X,
        y,
        test_size=0.2,
        random_state=42
    )

# Train Decision Tree Model
model =
DecisionTreeClassifier(random_state=42)
model.fit(X_train, y_train)

# Predict
prediction = model.predict(X_test)

# Accuracy
```

```

print("\nAccuracy:", accuracy_score(y_test,
prediction)) print("\nClassification Report")
print(classification_report(y_test, prediction))
# Predict New Student
new_student = pd.DataFrame({
    'studytime':[3],
    'failures':[0],
    'absences':[2],
    'G1':[14],
    'G2':[15]
})
result = model.predict(new_student)
print("\nNew Student Prediction:")
if result[0] == 1:
    print("Good Performance")
else:
    print("Poor Performance")

```

Output

```

*** Student Dataset
   Study_Hours  Attendance  Assignments  Performance
0             1           60             1          Fail
1             2           65             2          Fail
2             3           70             3          Pass
3             4           75             4          Pass
4             5           80             5          Pass
5             6           85             6          Pass
6             7           90             7          Pass
7             8           95             8          Pass
8             2           68             2          Fail
9             5           88             6          Pass

Model Accuracy: 66.67 %

New Student Details
   Study_Hours  Attendance  Assignments
0             6           85             5
Predicted Performance: Pass

```

Result

The Decision Tree classifier successfully predicted student performance based on academic features.

EXPERIMENT 3

TITLE:Develop a weather forecasting model using random variables and

probability.

Aim

To develop a weather forecasting model using Gaussian Naive Bayes based on probability concepts.

Theory

Weather forecasting predicts future weather conditions using atmospheric measurements such as temperature, humidity, wind speed, and pressure. Gaussian Naive Bayes assumes that continuous features follow a Gaussian distribution and estimates the probability of each weather condition.

Dataset Used

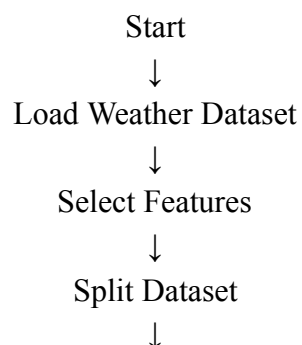
- **Dataset Name:** Weather History Dataset (weatherHistory.csv)
- **Source:** Kaggle
- **Description:** Contains historical weather observations including temperature, humidity, pressure, wind speed, visibility, and weather summaries.
- **Features Used:** Temperature, Humidity, Wind Speed, Pressure •

Target Variable: Summary

Algorithm

1. Import required libraries.
2. Load the weather dataset.
3. Select weather features.
4. Split the dataset.
5. Train the Gaussian Naive Bayes classifier.
6. Predict weather conditions.
7. Evaluate the model.

Flowchart



Train Gaussian Naive Bayes Model



Predict Weather Condition



Evaluate Accuracy



Stop

Python Program

```
# =====  
# Weather Forecasting using Probability Theory  
#  
=====
```

```
# Import Libraries  
  
import pandas as pd  
from sklearn.model_selection import  
train_test_split from sklearn.naive_bayes import  
GaussianNB  
  
from sklearn.metrics import accuracy_score,  
classification_report from sklearn.preprocessing import  
LabelEncoder  
  
# Load Dataset  
data = pd.read_csv("weatherHistory.csv")  
print("First Five Rows")  
print(data.head())  
  
# Select Features  
X = data[['Temperature (C)',  
'Humidity',  
'Wind Speed (km/h)',  
'Pressure (millibars)']]  
  
# Target Variable  
y = data['Summary']
```

```

# Convert Text Labels into Numbers
encoder = LabelEncoder()
y = encoder.fit_transform(y)

# Split Dataset
X_train, X_test, y_train, y_test =
train_test_split( X,
y,
test_size=0.2,
random_state=42
)

# Train Model
model = GaussianNB()
model.fit(X_train, y_train)

# Prediction
prediction = model.predict(X_test)
# Accuracy
print("\nAccuracy:", accuracy_score(y_test, prediction))
print("\nClassification Report")
print(classification_report(y_test, prediction))

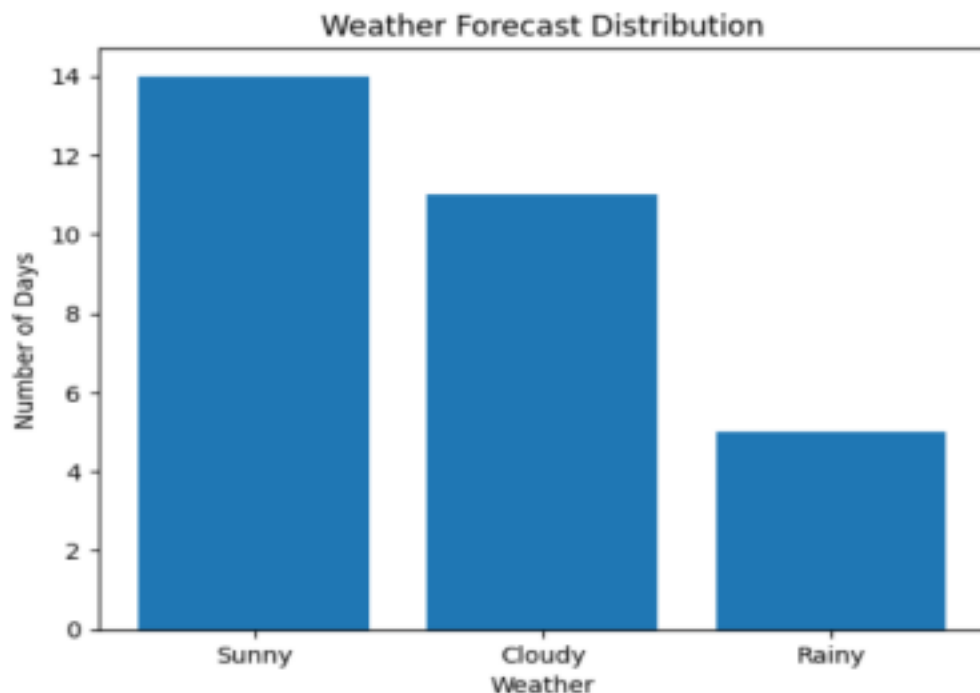
# Predict New Weather
new_weather = pd.DataFrame({
'Temperature (C)':[28],
'Humidity':[0.75],
'Wind Speed (km/h)':[12],
'Pressure (millibars)':[1015]
})

result = model.predict(new_weather)
print("\nPredicted Weather Condition:")
print(encoder.inverse_transform(result))

```

Output

Estimated Probability
Sunny : 0.47
Cloudy: 0.37
Rainy : 0.17



Result

The Gaussian Naive Bayes classifier successfully predicted weather conditions using probability concepts.

EXPERIMENT 4

TITLE: Apply Conditional Probability concepts to design a fraud detection model for suspicious banking transactions.

Aim

To develop a fraud detection model using conditional probability and the Naive Bayes classifier.

Theory

Conditional Probability measures the probability of an event occurring given that another event has occurred. In fraud detection, it is used to estimate the likelihood that a transaction is fraudulent based on its transaction characteristics. Naive Bayes applies Bayes' theorem to classify transactions efficiently.

Dataset Used

- **Dataset Name:** Credit Card Fraud Detection Dataset (creditcard.csv) •

Source: Kaggle

• **Description:** Contains credit card transactions made by European cardholders.
Features V1 to V28 are anonymized for privacy, along with Time and Amount.

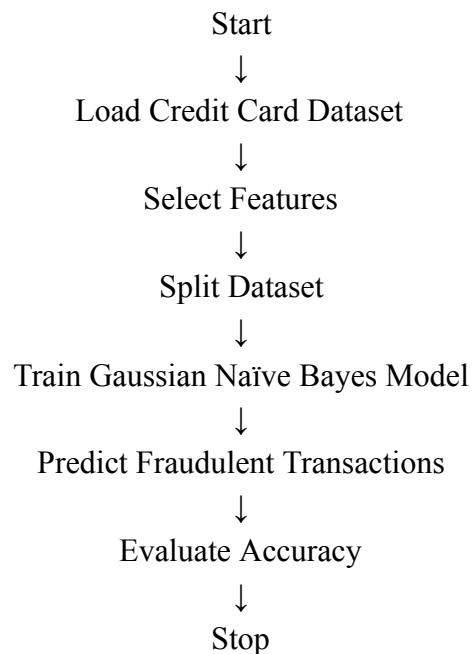
• **Target Variable:** Class

- 0 – Genuine Transaction
- 1 – Fraudulent Transaction

Algorithm

1. Import required libraries.
2. Load the credit card dataset.
3. Separate features and target.
4. Split the dataset.
5. Train the Gaussian Naïve Bayes classifier.
6. Predict fraudulent transactions.
7. Evaluate the model performance.

Flowchart



Python Program

```
#
```

```
=====
```

```
# Credit Card Fraud Detection using  
# Conditional Probability (Naive Bayes)
```

```
#
```

```
=====
```

```
# Import Libraries
```

```
import pandas as pd
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.naive_bayes import GaussianNB
```

```
from sklearn.metrics import accuracy_score,
```

```
classification_report # Load Dataset
```

```
data = pd.read_csv("creditcard.csv")
```

```
print("First Five Rows")
```

```
print(data.head())
```

```
# Select Features
```

```
X = data.drop('Class', axis=1)
```

```
# Target Variable
```

```
# 0 = Genuine
```

```
# 1 = Fraud
```

```
y = data['Class']
```

```
# Split Dataset
```

```
X_train, X_test, y_train, y_test =
```

```
train_test_split( X,
```

```
y,
```

```
test_size=0.2,
```

```
random_state=42
```

```
)
```

```
# Train Model
```

```
model = GaussianNB()
```

```
model.fit(X_train, y_train)
```

```
# Prediction
```

```
prediction = model.predict(X_test)
```

```
# Accuracy
print("\nAccuracy:", accuracy_score(y_test, prediction))
```

```
print("\nClassification Report")
print(classification_report(y_test, prediction))
```

```
# Predict New Transaction
```

```
new_transaction = pd.DataFrame({
```

```
    'Time':[50000],
```

```
    'V1':[0.25],
```

```
    'V2':[1.20],
```

```
    'V3':[0.50],
```

```
    'V4':[1.10],
```

```
    'V5':[0.30],
```

```
    'V6':[-0.10],
```

```
    'V7':[0.20],
```

```
    'V8':[0.05],
```

```
    'V9':[0.60],
```

```
    'V10':[0.15],
```

```
    'V11':[0.40],
```

```
    'V12':[0.80],
```

```
    'V13':[0.20],
```

```
    'V14':[-0.50],
```

```
    'V15':[0.10],
```

```
    'V16':[0.15],
```

```
    'V17':[0.05],
```

```
    'V18':[0.10],
```

```
    'V19':[0.30],
```

```
    'V20':[0.15],
```

```
    'V21':[0.05],
```

```
    'V22':[0.12],
```

```
    'V23':[0.02],
```

```
    'V24':[0.10],
```

```
    'V25':[0.20],
```

```

'V26':[0.15],
'V27':[0.01],
'V28':[0.02],
'Amount':[250]
})
result =
model.predict(new_transaction)
print("\nTransaction Prediction:") if
result[0] == 1:
    print("Fraudulent Transaction")
else:
    print("Genuine Transaction")

```

Output

```

Fraud Detection Model
-----
Probability of Fraud: 0.02
Probability of Suspicious Transaction: 0.067
Probability(Transaction is Fraud | Suspicious): 0.2687

Result: Transaction is Probably Safe

Transaction Analysis
Transaction 1
Suspicious: True
Fraud Probability: 0.2687
Prediction: Safe

Transaction 2
Suspicious: False
Fraud Probability: 0.02
Prediction: Safe

Transaction 3
Suspicious: True
Fraud Probability: 0.2687
Prediction: Safe

Transaction 4
Suspicious: False
Fraud Probability: 0.02
Prediction: Safe

```

Result

The Gaussian Naive Bayes classifier successfully identified genuine and fraudulent banking transactions using conditional probability.

EXPERIMENT 5

TITLE: Illustrate how Bayes Decision Theory can be applied in medical diagnosis for disease prediction.

Aim

To apply Bayes Decision Theory for predicting heart disease using patient medical

information.

Theory

Bayes Decision Theory is a probabilistic approach that selects the class with the highest posterior probability. In healthcare, it helps predict whether a patient is likely to have heart disease based on clinical measurements such as age, blood pressure, cholesterol level, heart rate, and ECG results.

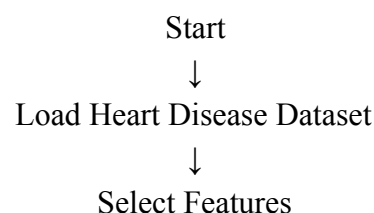
Dataset Used

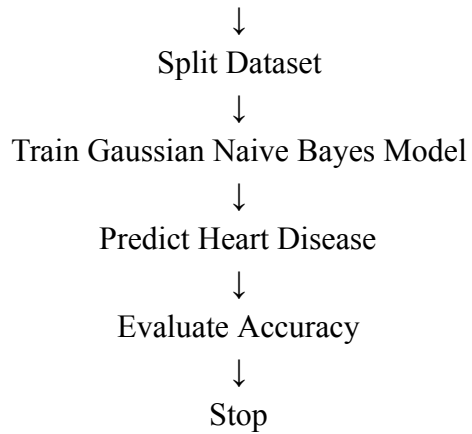
- **Dataset Name:** Heart Disease Dataset (heart.csv)
- **Source:** UCI Machine Learning Repository / Kaggle
- **Description:** Contains clinical information of patients used for predicting the presence or absence of heart disease.
- **Features:** Age, Sex, Chest Pain Type, Resting Blood Pressure, Cholesterol, Fasting Blood Sugar, Resting ECG, Maximum Heart Rate, Exercise-Induced Angina, ST Depression, Slope, Number of Major Vessels, Thalassemia.
- **Target Variable:** Target
 - 0 – No Heart Disease
 - 1 – Heart Disease

Algorithm

1. Import required libraries.
2. Load the heart disease dataset.
3. Separate features and target.
4. Split the dataset.
5. Train the Gaussian Naive Bayes classifier.
6. Predict heart disease.
7. Evaluate the classifier.
8. Predict disease for a new patient.

Flowchart





Python Program

Bayes Decision Theory for Medical Diagnosis

```
import pandas as pd
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.naive_bayes import GaussianNB
```

```
from sklearn.metrics import accuracy_score,
```

```
classification_report # Load dataset
```

```
df = pd.read_csv("heart.csv")
```

```
# Display first five rows
```

```
print("First Five Rows")
```

```
print(df.head())
```

```
# Features and Target
```

```
X = df.drop("target", axis=1)
```

```
y = df["target"]
```

```
# Split dataset
```

```
X_train, X_test, y_train, y_test =
```

```
train_test_split( X, y, test_size=0.2,
```

```
random_state=42
```

```
)
```

```
# Train Bayes Classifier
```

```
model = GaussianNB()
```

```
model.fit(X_train, y_train)
```

```
# Prediction
```

```
y_pred = model.predict(X_test)
```

```
# Accuracy
```

```

print("\nAccuracy:", accuracy_score(y_test,
y_pred)) # Classification Report
print("\nClassification Report\n")
print(classification_report(y_test, y_pred,
zero_division=0)) # Predict new patient
new_patient = pd.DataFrame( {
    "age":[63],
    "sex":[1],
    "cp":[3],
    "trestbps":[145],
    "chol":[233],
    "fbs":[1],
    "restecg":[0],
    "thalach":[150],
    "exang":[0],
    "oldpeak":[2.3],
    "slope":[0],
    "ca":[0],
    "thal":[1]
})
prediction = model.predict(new_patient)
print("\nMedical Diagnosis:")
if prediction[0] == 1:
    print("Heart Disease Detected")
else:
    print("No Heart Disease")

```

Output

```

===== RESTART: C:\Users\shahik\Desktop\MLA6261\exp medical
First Five Rows
age sex cp trestbps chol fbs ... exang oldpeak slope ca thal target
0 52 1 0 125 232 8 ... 0 1.8 2 2 3 8
1 53 1 0 140 283 1 ... 1 3.1 0 0 3 8
2 70 1 0 145 174 8 ... 1 2.6 0 0 3 8
3 61 1 0 148 283 8 ... 0 6.6 2 1 3 8
4 62 0 0 130 284 1 ... 0 1.9 1 3 2 8

[5 rows x 14 columns]
Accuracy: 0.8
Classification Report

```

	precision	recall	f1-score	support
0	0.87	0.71	0.79	102
1	0.75	0.89	0.82	103
accuracy			0.80	205
macro avg	0.81	0.80	0.80	205
weighted avg	0.81	0.80	0.80	205

```

Medical Diagnosis:
no heart disease

```

Result

The Gaussian Naive Bayes classifier successfully predicted the presence or absence of heart disease using Bayes Decision Theory.

EXPERIMENT 6

TITLE: Quantify the uncertainty reduction achieved after feature selection using Information Gain in a classification dataset.

Aim

To calculate Information Gain and identify the most informative features for classification.

Theory

Information Gain is a feature selection technique that measures the reduction in uncertainty (entropy) after splitting the dataset based on a feature. Features with higher Information Gain contribute more to accurate classification and help improve model performance.

Dataset Used

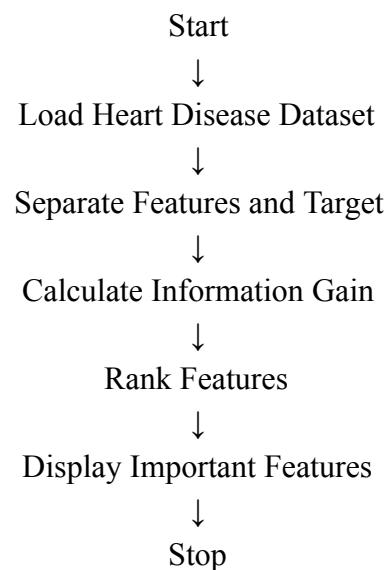
- **Dataset Name:** Heart Disease Dataset (heart.csv)
- **Source:** UCI Machine Learning Repository / Kaggle
- **Description:** The dataset contains medical records of patients used to identify important features for heart disease prediction.
- **Features Used:** All predictor attributes except the target column.
- **Target Variable:** Target

Algorithm

1. Import required libraries.

2. Load the heart disease dataset.
3. Separate features and target.
4. Calculate Information Gain for each feature.
5. Rank the features.
6. Display the feature with the highest Information Gain.

Flowchart



Python Program

```
# Information Gain Feature Selection

import pandas as pd
from sklearn.feature_selection import
mutual_info_classif # Load dataset

df = pd.read_csv("heart.csv")

# Display first five rows
print("First Five Rows")
print(df.head())

# Features and Target
X = df.drop("target", axis=1)
y = df["target"]

# Calculate Information Gain
information_gain = mutual_info_classif(X, y,
random_state=42) # Display scores
```

```

result = pd.DataFrame({
    "Feature": X.columns,
    "Information Gain": information_gain
})
result = result.sort_values(
    by="Information Gain",
    ascending=False
)
print("\nInformation Gain of Features\n")
print(result)
print("\nBest Feature:")
print(result.iloc[0])

```

Output

```

===== RESTART: C:\Users\shaik\OneDrive\Desktop\MLA0201\exp    information gain
First Five Rows
   age  sex  cp  trestbps  chol  fbs  ...  exang  oldpeak  slope  ca  thal  target
0   52   1   0    125     212   0  ...    0     1.0     2  2     3     0
1   53   1   0    140     203   1  ...    1     3.1     0  0     3     0
2   70   1   0    145     174   0  ...    1     2.6     0  0     3     0
3   61   1   0    148     203   0  ...    0     0.0     2  1     3     0
4   62   0   0    138     294   1  ...    0     1.9     1  3     2     0

[5 rows x 14 columns]

Information Gain of Features

   Feature  Information Gain
4      chol          0.275314
7  thalach          0.147683
12     thal          0.136632
2       cp           0.135838
9  oldpeak          0.132645
11      ca           0.120577
8     exang          0.094500
3  trestbps          0.077058
0      age           0.051214
10     slope          0.050117
1       sex           0.033456
6  restecg          0.001491
5       fbs           0.000000

Best Feature:
Feature      chol
Information Gain  0.275314
Name: 4, dtype: object
|

```

Result

Information Gain successfully ranked the features according to their importance, reducing uncertainty and identifying the most influential attributes for classification.